

ApiGenie — Admin Guide

Advanced configuration: detection rules, S1 Detection Library integration, custom listeners, HEC transport details, profile tuning, and system settings.

1. Detection Rules

Detection rules override specific fields in normal generated logs to trigger SIEM detection rules.

Create a detection rule

Log Profiles & Detection Rules → expand ▶ Detection Rules → + New Rule.

Field	Description	Example
Name	Rule identifier	LSASS Access Detection
Source	Which source's logs to inject into	sentinelone
Enabled	Toggle on/off	✓
Periodicity	1 in N logs (≤100) or every N seconds (>100)	10 = 1 in every 10 logs
Field overrides	Key-value pairs replacing fields	event.type = Process Access , tgt.process.name = lsass.exe

Edit Detection Rule

Name

Brute force login

Source

Okta

Description

Triggers brute force detection

Periodicity

5

1-100 = every N-th log; >100 = every N seconds

Enabled ☒

Field Overrides (dot notation for nested fields, e.g. outcome.result)

eventType	user.session.start	x
outcome.result	FAILURE	x
severity	WARN	x
displayMessage	Authentication failed - invalid credentials	x

+ Add field

Cancel

Delete

Save

How injection works

When a pull source generates logs or a push source sends events, the detection rules engine checks:

1. Is the rule enabled and does the source match?
2. Has the periodicity threshold been reached?
3. If yes → override the specified fields in the next generated log

This means the log appears normal except for the overridden fields — exactly like a real malicious event mixed into normal traffic.

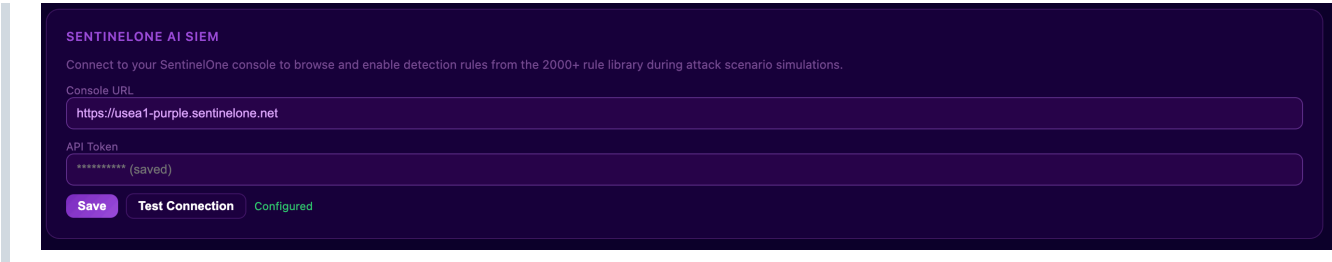
2. S1 Detection Library Integration

Prerequisites

Configure S1 console access in **System Settings**:

Setting	Value
Console URL	<code>https://usea1-purple.sentinelone.net</code> (your console)
API Token	S1 API token with detection rule read/write scope

Click **Test Connection** to verify.

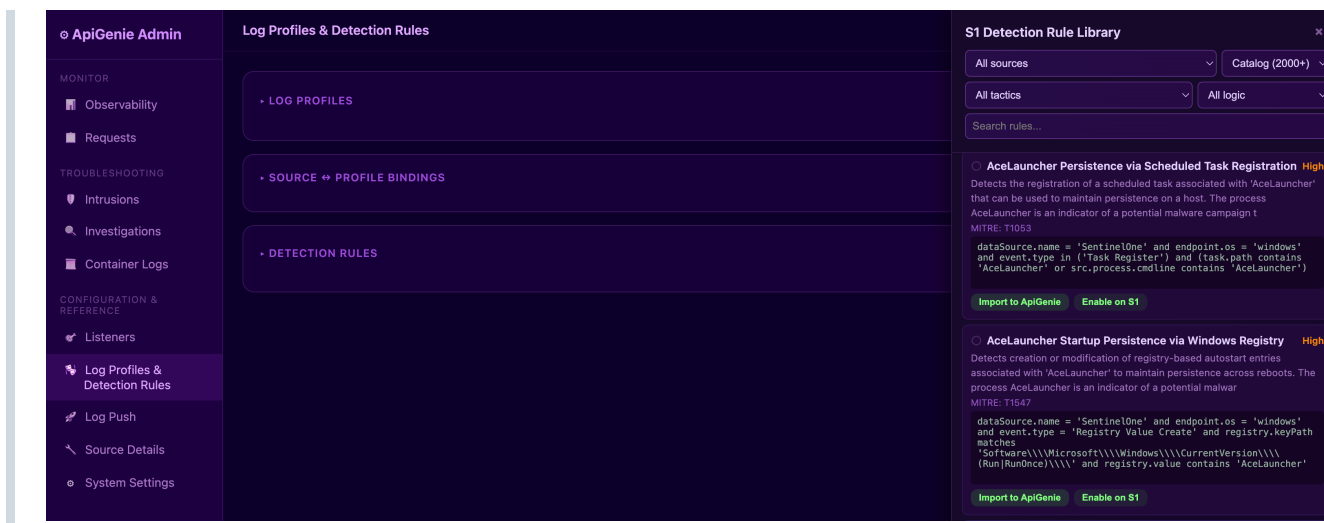


Browse rules

Log Profiles & Detection Rules → **Browse S1 Library** button.

A 480px slide-out drawer opens with filters:

Filter	Options
Source	All sources, or pick one (Okta, SentinelOne, Palo Alto, etc.)
Type	Catalog (2000+) or Custom rules
Tactic	All tactics, or a specific MITRE ATT&CK tactic
Logic	All logic / Visible (importable) / Hidden (browse only)
Search	Free-text search across rule names



Rule card anatomy

Each rule card shows:

- **Status indicator** — ● enabled (green) / ○ disabled (grey)
- **Name** — rule title
- **Severity** — Critical (red), High (orange), Medium (yellow), Low (grey)
- **Description** — full text, no truncation
- **MITRE** — technique IDs (e.g., T1547, T1059.001)
- **Detection logic** — full s1ql query in monospace, scrollable, word-wrapped
- **Action buttons** — Import to ApiGenie / Enable on S1

Import a rule

Click **Import to ApiGenie**. A preview dialog shows:

- Rule name (prefixed with [S1])
- Auto-detected source
- Field overrides extracted from the s1ql query

Field mapping logic:

- **S1 native fields** (`endpoint.os` , `event.type` , `src.process.cmdline` , `registry.keyPath` , etc.) → passed through as-is (type: `native`)
- **unmapped.* fields** → prefix stripped (type: `unmapped`)
- **OCSF fields** → reverse-mapped via per-source lookup table (type: `mapped`)
- **Operators parsed:** `=` , `==` , `contains` , `ContainsCIS` , `matches` , `startswith` , `endswith` , `in (...)`

Crafted with ♥ by the RoarinPenguin

Click **OK** to create the local detection rule with pre-populated field overrides.

localhost says

Import "[S1] AceLauncher Startup Persistence via Windows Registry" as a local detection rule?

Source: sentinelone

Field overrides:

- endpoint.os = windows (native)
- event.type = Registry Value Create (native)
- registry.keyPath = Software\\\\\\\\Microsoft\\\\\\\\Windows\\\\\\\\CurrentVersion\\\\\\\\(Run|RunOnce)\\\\\\\\ (native)

Cancel

OK

Enable/disable rules on S1

Click **Enable on S1** or **Disable on S1** to toggle a rule's status directly on your SentinelOne console via API. This requires a token with write scope.

Logic visibility

- **Visible** rules have s1ql queries exposed in the API response → importable
- **Hidden** rules have no query content (typically "First Seen" behavioral rules) → browse-only
- When selecting the "Visible" filter, the type auto-switches to match

3. Log Push — Advanced Configuration

HEC flavour details

Flavour	Auth header	Extra headers	Endpoint path
Splunk HEC	Authorization: Splunk <token>	—	/services/collector/event or /services/collector/raw
S1 AI SIEM	Authorization: Bearer <api-token>	S1-Scope: account=<id>	/services/collector/raw
Observe / S1 DPM	Authorization: Bearer <jwt>	—	/services/collector/raw

Auto-detection:

- If the host contains `sentinelone.net` and the path is HEC-like → S1 AI SIEM
- If the stored profile has `hec_flavour: observe` → Observe (uses HTTP/2 via `http.client`)
- Otherwise → Splunk HEC

Syslog configuration

Parameter	Description
Protocol	TCP or UDP
Host:Port	Syslog server address
Facility	0–23 (default: 1 = user)
Format	RFC 5424

HTTP POST configuration

Parameter	Description
URL	Full destination URL
Headers	Custom headers (JSON object)
TLS	Enable/disable

4. Log Profiles — Advanced

Signal-to-noise ratio

Controls how often profile entities appear in generated logs:

- **100%** — every log uses profile entities
- **50%** — half use profile entities, half use random
- **10%** — rare profile entities, mostly random

Log volume scaling

Controls the number of events returned per API call:

- **100%** — full output (e.g., 50 events per request)
- **25%** — quarter output (12–13 events per request)
- Useful for simulating low-traffic sources

Entity types

Entity	Fields	Used by
Users	username, domain, email, department, title	All sources with user context
Machines	hostname, IP, OS, workstation name	EDR, network sources
C2 servers	FQDN, IP, port, protocol	Threat-related sources
Malware	filename, family, hash, cmdline	EDR, threat sources
Mail senders	from, to, subject, attachment	Email security sources

5. Custom Listeners

Create custom HTTP endpoints that accept any payload.

Listeners tab → **+ New Listener**.

Field	Description
ID	URL path segment (e.g., <code>my-webhook</code>)
Auth	None, Bearer, Basic, API key
Response status	HTTP status code to return
Response body	Static JSON response
Chaos mode	Random failures at configurable rate
Rate limit	Max requests per minute

Access at: `https://<domain>/listener/<id>/<any-path>`

Each listener has its own request log in the admin UI.

6. Intrusion Detection

The **Intrusions** tab aggregates suspicious patterns detected in incoming requests — SQL injection, XSS, path traversal, etc. Useful for testing WAF rule effectiveness.

7. Investigations

The **Investigations** tab provides a guided triage workflow for exploring generated events across sources.

8. Container Logs

The **Container Logs** tab streams live logs from all Docker containers (apigenie, nginx, kafka, zookeeper, pubsub) in the admin UI.

9. System Settings

Setting	Description
S1 Console URL	SentinelOne management console URL
S1 API Token	Token for Detection Library API access
Test Connection	Verifies API connectivity and permissions

10. Data Persistence

All configuration is stored as JSON files in `$APIGENIE_DATA_ROOT` (default: `/data/` inside the container, mounted as a Docker volume).

File	Contents
<code>detection_rules.json</code>	Local detection rules
<code>push_profiles.json</code>	Log push configurations
<code>profiles.json</code>	Log profiles (entity pools)
<code>bindings.json</code>	Source ↔ profile assignments
<code>s1_settings.json</code>	S1 console URL + token
<code>listeners.json</code>	Custom listener configs

11. API Reference — Admin Endpoints

All admin API endpoints are prefixed with `/admin/api/` and require session auth.

Detection Rules

GET	<code>/admin/api/detection-rules</code>	– list all rules
POST	<code>/admin/api/detection-rules</code>	– create rule
PUT	<code>/admin/api/detection-rules/{id}</code>	– update rule
DELETE	<code>/admin/api/detection-rules/{id}</code>	– delete rule

S1 Integration

GET	/admin/api/s1/settings	– get S1 console settings
POST	/admin/api/s1/settings	– save S1 console settings
POST	/admin/api/s1/test	– test S1 connection
GET	/admin/api/s1/rules	– query S1 catalog rules
GET	/admin/api/s1/custom-rules	– query S1 custom rules
GET	/admin/api/s1/rules/{id}/import-preview	– preview import mapping
POST	/admin/api/s1/rules/import	– import rule as local
PUT	/admin/api/s1/rules/{id}/enable	– enable rule on S1
PUT	/admin/api/s1/rules/{id}/disable	– disable rule on S1

Profiles & Push

GET	/admin/api/profiles	– list profiles
POST	/admin/api/profiles	– create profile
PUT	/admin/api/profiles/{id}	– update profile
DELETE	/admin/api/profiles/{id}	– delete profile
GET	/admin/api/bindings	– list source bindings
POST	/admin/api/bindings	– save binding
GET	/admin/api/push-profiles	– list push profiles
POST	/admin/api/push-profiles	– create push profile
POST	/admin/api/push/{id}/start	– start pushing
POST	/admin/api/push/{id}/stop	– stop pushing